Guía de Laboratorio: Pokédex

Kotlin + Jetpack Compose + PokéAPI

Objetivo

Construir una aplicación Android que consuma la PokéAPI para mostrar una lista de Pokémon (con buscador) y su detalle con datos en español, aplicando arquitectura por capas (data - domain - ui) con Jetpack Compose, Retrofit y Navigation.

Requisitos previos

- Android Studio (Ladybug o superior)
- SDK mínimo: API 26 (Android 8.0)
- Conexión a Internet

Estructura del proyecto

```
com.carevalojesus.pokeapi
 ├─ data
 │   ├─ remote
 │   │   ├─ dto
 │   │   │   ├─ PokemonListResponse.kt
 │   │   │   ├─ PokemonDetailResponse.kt
 │   │   │   └─ PokemonSpeciesResponse.kt
 │   │   ├─ PokemonApi.kt
 │   │   └─ RetrofitClient.kt
 │   └─ repository
 │       └─ PokemonRepository.kt
 ├─ domain
 │   └─ model
 │       ├─ PokemonItem.kt
 │       └─ PokemonDetail.kt
 ├─ ui
 │   ├─ navigation
 │   │   └─ AppNav.kt
 │   ├─ screens
 │   │   ├─ pokedex
 │   │   │   ├─ PokedexViewModel.kt
 │   │   │   ├─ PokedexUiState.kt
 │   │   │   └─ PokedexScreen.kt
 │   │   └─ detail
 │   │       ├─ PokemonDetailViewModel.kt
 │   │       ├─ PokemonDetailUiState.kt
 │   │       └─ PokemonDetailScreen.kt
 │   └─ theme/
 └─ MainActivity.kt
```

Descripción de capas

Capa	Responsabilidad
data/remote/dto	Data Transfer Objects: mapean exactamente el JSON de la API
data/remote	Interfaz Retrofit (`PokemonApi`) y singleton de configuración (`RetrofitClient`)
data/repository	Orquesta llamadas a la API, traduce datos al español y transforma DTOs en modelos de dominio
domain/model	Modelos limpios que usa la UI, sin dependencias de red
ui/screens	Pantallas Compose, ViewModels y estados de UI
ui/navigation	Grafo de navegación con Navigation Compose

Paso 1: Agregar dependencias

1.1 Catálogo de versiones (`gradle/libs.versions.toml`)

Agregar las siguientes entradas:

```
[versions]
# ... versiones existentes ...
retrofit = "2.11.0"
coilCompose = "2.7.0"
navigationCompose = "2.8.4"

[libraries]
# ... librerías existentes ...
androidx-lifecycle-viewmodel-compose = { group = "androidx.lifecycle",
    name = "lifecycle-viewmodel-compose", version.ref = "lifecycleRuntimeKtx" }
retrofit = { group = "com.squareup.retrofit2",
    name = "retrofit", version.ref = "retrofit" }
retrofit-converter-gson = { group = "com.squareup.retrofit2",
    name = "converter-gson", version.ref = "retrofit" }
coil-compose = { group = "io.coil-kt",
    name = "coil-compose", version.ref = "coilCompose" }
androidx-navigation-compose = { group = "androidx.navigation",
    name = "navigation-compose", version.ref = "navigationCompose" }
```

1.2 Build del módulo app (`app/build.gradle.kts`)

Agregar al bloque `dependencies` :

```
dependencies {
    // ... existentes ...
    implementation(libs.androidx.lifecycle.viewmodel.compose)
    // Retrofit + Gson
    implementation(libs.retrofit)
    implementation(libs.retrofit.converter.gson)
```

```
    // Coil (carga de imágenes)
    implementation(libs.coil.compose)
    // Navigation Compose
    implementation(libs.androidx.navigation.compose)
}
```

1.3 Permiso de Internet (`AndroidManifest.xml`)

Agregar **antes** de `<application>` :

```
<uses-permission android:name="android.permission.INTERNET" />
```

> ***Sync Now*** *en Android Studio después de estos cambios.*

Paso 2: Capa de datos - DTOs

Los DTOs mapean la respuesta JSON de la PokéAPI a clases Kotlin.

2.1 `data/remote/dto/PokemonListResponse.kt`

```
package com.carevalojesus.pokeapi.data.remote.dto

data class PokemonListResponse(
    val count: Int,
    val results: List<PokemonListEntry>
)

data class PokemonListEntry(
    val name: String,
    val url: String
)
```

Mapea la respuesta de `GET /api/v2/pokemon?limit=151` . Cada `PokemonListEntry` contiene el nombre y la URL del recurso.

2.2 `data/remote/dto/PokemonDetailResponse.kt`

```
package com.carevalojesus.pokeapi.data.remote.dto

import com.google.gson.annotations.SerializedName

data class PokemonDetailResponse(
    val id: Int,
    val name: String,
    val height: Int,
    val weight: Int,
    val types: List<TypeSlot>,
    val sprites: Sprites,
    val stats: List<StatSlot>
```

```
)

data class TypeSlot(
    val slot: Int,
    val type: TypeInfo
)

data class TypeInfo(
    val name: String
)

data class Sprites(
    @SerializedName("front_default")
    val frontDefault: String?,
    val other: OtherSprites?
)

data class OtherSprites(
    @SerializedName("official-artwork")
    val officialArtwork: OfficialArtwork?
)

data class OfficialArtwork(
    @SerializedName("front_default")
    val frontDefault: String?
)

data class StatSlot(
    @SerializedName("base_stat")
    val baseStat: Int,
    val stat: StatInfo
)

data class StatInfo(
    val name: String
)
```

Mapea la respuesta de `GET /api/v2/pokemon/{id}`. La anotación `@SerializedName` se usa cuando el nombre del campo en JSON difiere de Kotlin (ej: `front_default` a `frontDefault`).

2.3 `data/remote/dto/PokemonSpeciesResponse.kt`

```
package com.carevalojesus.pokeapi.data.remote.dto

import com.google.gson.annotations.SerializedName

data class PokemonSpeciesResponse(
    val names: List<SpeciesName>,
    val genera: List<Genus>,
    @SerializedName("flavor_text_entries")
    val flavorTextEntries: List<FlavorTextEntry>
```

```
)

data class SpeciesName(
    val name: String,
    val language: Language
)

data class Genus(
    val genus: String,
    val language: Language
)

data class FlavorTextEntry(
    @SerializedName("flavor_text")
    val flavorText: String,
    val language: Language
)

data class Language(
    val name: String
)
```

Mapea la respuesta de `GET /api/v2/pokemon-species/{id}`. Este endpoint proporciona los datos localizados: nombre en español, categoría (genus) y descripción de la Pokédex (flavor text) en múltiples idiomas.

Paso 3: Capa de datos - API y Retrofit

3.1 `data/remote/PokemonApi.kt`

```
package com.carevalojesus.pokeapi.data.remote

import com.carevalojesus.pokeapi.data.remote.dto.PokemonDetailResponse
import com.carevalojesus.pokeapi.data.remote.dto.PokemonListResponse
import com.carevalojesus.pokeapi.data.remote.dto.PokemonSpeciesResponse
import retrofit2.http.GET
import retrofit2.http.Path
import retrofit2.http.Query

interface PokemonApi {

    @GET("pokemon")
    suspend fun getPokemonList(
        @Query("limit") limit: Int = 151,
        @Query("offset") offset: Int = 0
    ): PokemonListResponse

    @GET("pokemon/{id}")
    suspend fun getPokemonDetail(
        @Path("id") id: Int
    ): PokemonDetailResponse
```

```
    @GET("pokemon-species/{id}")
    suspend fun getPokemonSpecies(
        @Path("id") id: Int
    ): PokemonSpeciesResponse
}
```

Conceptos clave:

- `@GET` define el endpoint relativo a la URL base.
- `@Query` agrega parámetros de consulta (`?limit=151&offset=0`).
- `@Path` reemplaza un segmento de la ruta (`{id}`).
- `suspend` permite llamar estas funciones desde coroutines.
- Se usan 3 endpoints: lista, detalle técnico y especie (datos localizados).

3.2 `data/remote/RetrofitClient.kt`

```
package com.carevalojesus.pokeapi.data.remote

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitClient {

    private const val BASE_URL = "https://pokeapi.co/api/v2/"

    val api: PokemonApi by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(PokemonApi::class.java)
    }
}
```

Conceptos clave:

- `object` crea un singleton en Kotlin.
- `by lazy` inicializa Retrofit solo cuando se usa por primera vez.
- `GsonConverterFactory` convierte automáticamente JSON a data classes.

Paso 4: Capa de dominio - Modelos

Los modelos de dominio son independientes de la API. Solo contienen los datos que la UI necesita, ya traducidos al español.

4.1 `domain/model/PokemonItem.kt`

```
package com.carevalojesus.pokeapi.domain.model
```

```
data class PokemonItem(
    val id: Int,
    val name: String,
    val imageUrl: String
)
```

Se usa en la pantalla de lista (Pokédex).

4.2 `domain/model/PokemonDetail.kt`

```
package com.carevalojesus.pokeapi.domain.model

data class PokemonDetail(
    val id: Int,
    val name: String,
    val height: Int,
    val weight: Int,
    val types: List<String>,
    val imageUrl: String,
    val stats: List<Stat>,
    val genus: String,
    val description: String
)

data class Stat(
    val name: String,
    val value: Int
)
```

Se usa en la pantalla de detalle. Incluye `genus` (categoría, ej: "Pokémon Semilla") y `description` (texto de la Pokédex en español).

Paso 5: Capa de datos - Repositorio

5.1 `data/repository/PokemonRepository.kt`

```
package com.carevalojesus.pokeapi.data.repository

import com.carevalojesus.pokeapi.data.remote.RetrofitClient
import com.carevalojesus.pokeapi.domain.model.PokemonDetail
import com.carevalojesus.pokeapi.domain.model.PokemonItem
import com.carevalojesus.pokeapi.domain.model.Stat
import kotlinx.coroutines.async
import kotlinx.coroutines.coroutineScope

class PokemonRepository {

    private val api = RetrofitClient.api
```

```
private val typeTranslations = mapOf(
    "normal" to "Normal",
    "fire" to "Fuego",
    "water" to "Agua",
    "electric" to "Eléctrico",
    "grass" to "Planta",
    "ice" to "Hielo",
    "fighting" to "Lucha",
    "poison" to "Veneno",
    "ground" to "Tierra",
    "flying" to "Volador",
    "psychic" to "Psíquico",
    "bug" to "Bicho",
    "rock" to "Roca",
    "ghost" to "Fantasma",
    "dragon" to "Dragón",
    "dark" to "Siniestro",
    "steel" to "Acero",
    "fairy" to "Hada"
)

private val statTranslations = mapOf(
    "hp" to "PS",
    "attack" to "Ataque",
    "defense" to "Defensa",
    "special-attack" to "At. Esp.",
    "special-defense" to "Def. Esp.",
    "speed" to "Velocidad"
)

suspend fun getPokemonList(): List<PokemonItem> {
    val response = api.getPokemonList()
    return response.results.map { entry ->
        val id = entry.url.trimEnd('/').split("/").last().toInt()
        PokemonItem(
            id = id,
            name = entry.name.replaceFirstChar { it.uppercase() },
            imageUrl = "https://raw.githubusercontent.com/PokeAPI/" +
                "sprites/master/sprites/pokemon/" +
                "other/official-artwork/$id.png"
        )
    }
}

suspend fun getPokemonDetail(id: Int): PokemonDetail =
    coroutineScope {
        val detailDeferred = async { api.getPokemonDetail(id) }
        val speciesDeferred = async { api.getPokemonSpecies(id) }

        val response = detailDeferred.await()
        val species = speciesDeferred.await()
```

```
            val spanishName = species.names
                .firstOrNull { it.language.name == "es" }?.name
                ?: response.name.replaceFirstChar { it.uppercase() }

            val genus = species.genera
                .firstOrNull { it.language.name == "es" }?.genus
                ?: ""

            val description = species.flavorTextEntries
                .lastOrNull { it.language.name == "es" }
                ?.flavorText
                ?.replace("\n", " ")
                ?.replace("\u000c", " ")
                ?: ""

            PokemonDetail(
                id = response.id,
                name = spanishName,
                height = response.height,
                weight = response.weight,
                types = response.types.map { typeSlot ->
                    typeTranslations[typeSlot.type.name]
                        ?: typeSlot.type.name
                            .replaceFirstChar { it.uppercase() }
                },
                imageUrl = response.sprites.other
                    ?.officialArtwork?.frontDefault
                    ?: response.sprites.frontDefault ?: "",
                stats = response.stats.map { statSlot ->
                    Stat(
                        name = statTranslations[statSlot.stat.name]
                            ?: statSlot.stat.name,
                        value = statSlot.baseStat
                    )
                },
                genus = genus,
                description = description
            )
        }
}
```

Conceptos clave:

- **Traducciones locales**: `typeTranslations` (18 tipos) y `statTranslations` (6 stats) convierten los nombres del inglés al español sin llamadas adicionales a la API.
- **Llamadas paralelas con `async`**: `getPokemonDetail` ejecuta las peticiones a `pokemon/{id}` y `pokemon-species/{id}` simultáneamente dentro de `coroutineScope`, reduciendo el tiempo de espera.
- **Datos en español desde la API**: El nombre, la categoría (`genus`) y la descripción se extraen del endpoint `pokemon-species` filtrando por `language.name == "es"` .
- **Limpieza de texto**: La descripción puede contener saltos de línea y caracteres de control (`\u000c`), que se reemplazan por espacios.

Paso 6: Capa UI - Estados

Los estados representan las tres posibles situaciones de cada pantalla: cargando, éxito o error.

6.1 `ui/screens/pokedex/PokedexUiState.kt`

```
package com.carevalojesus.pokeapi.ui.screens.pokedex

import com.carevalojesus.pokeapi.domain.model.PokemonItem

sealed interface PokedexUiState {
    data object Loading : PokedexUiState
    data class Success(
        val pokemonList: List<PokemonItem>
    ) : PokedexUiState
    data class Error(val message: String) : PokedexUiState
}
```

6.2 `ui/screens/detail/PokemonDetailUiState.kt`

```
package com.carevalojesus.pokeapi.ui.screens.detail

import com.carevalojesus.pokeapi.domain.model.PokemonDetail

sealed interface PokemonDetailUiState {
    data object Loading : PokemonDetailUiState
    data class Success(
        val pokemon: PokemonDetail
    ) : PokemonDetailUiState
    data class Error(val message: String) : PokemonDetailUiState
}
```

¿Por qué `sealed interface`? Garantiza que el `when` en Compose sea exhaustivo: el compilador verifica que se manejen todos los casos.

Paso 7: Capa UI - ViewModels

7.1 `ui/screens/pokedex/PokedexViewModel.kt`

```
package com.carevalojesus.pokeapi.ui.screens.pokedex

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.carevalojesus.pokeapi.data.repository.PokemonRepository
import com.carevalojesus.pokeapi.domain.model.PokemonItem
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch
```

```
class PokedexViewModel : ViewModel() {

    private val repository = PokemonRepository()

    private var allPokemon: List<PokemonItem> = emptyList()

    private val _uiState =
        MutableStateFlow<PokedexUiState>(PokedexUiState.Loading)
    val uiState: StateFlow<PokedexUiState> = _uiState

    private val _searchQuery = MutableStateFlow("")
    val searchQuery: StateFlow<String> = _searchQuery

    init {
        loadPokemonList()
    }

    fun loadPokemonList() {
        viewModelScope.launch {
            _uiState.value = PokedexUiState.Loading
            try {
                allPokemon = repository.getPokemonList()
                filterPokemon(_searchQuery.value)
            } catch (e: Exception) {
                _uiState.value = PokedexUiState.Error(
                    e.message ?: "Error desconocido"
                )
            }
        }
    }

    fun onSearchQueryChange(query: String) {
        _searchQuery.value = query
        filterPokemon(query)
    }

    private fun filterPokemon(query: String) {
        if (query.isBlank()) {
            _uiState.value = PokedexUiState.Success(allPokemon)
        } else {
            val filtered = allPokemon.filter { pokemon ->
                pokemon.name.contains(query, ignoreCase = true)
                    || pokemon.id.toString() == query
            }
            _uiState.value = PokedexUiState.Success(filtered)
        }
    }
}
```

Conceptos clave del buscador:

- `allPokemon` : Lista completa en memoria que sirve como fuente para el filtrado.
- `searchQuery` : `StateFlow<String>` que expone el texto actual de la barra de búsqueda.
- `onSearchQueryChange()` : Se llama cada vez que el usuario escribe; actualiza el query y filtra.
- `filterPokemon()` : Filtra por nombre (parcial, case-insensitive) o por número exacto del Pokémon.

7.2 `ui/screens/detail/PokemonDetailViewModel.kt`

```
package com.carevalojesus.pokeapi.ui.screens.detail

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.carevalojesus.pokeapi.data.repository.PokemonRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch

class PokemonDetailViewModel : ViewModel() {

    private val repository = PokemonRepository()

    private val _uiState =
        MutableStateFlow<PokemonDetailUiState>(
            PokemonDetailUiState.Loading
        )
    val uiState: StateFlow<PokemonDetailUiState> = _uiState

    fun loadPokemon(id: Int) {
        viewModelScope.launch {
            _uiState.value = PokemonDetailUiState.Loading
            try {
                val detail = repository.getPokemonDetail(id)
                _uiState.value =
                    PokemonDetailUiState.Success(detail)
            } catch (e: Exception) {
                _uiState.value = PokemonDetailUiState.Error(
                    e.message ?: "Error desconocido"
                )
            }
        }
    }
}
```

Conceptos clave:

- `MutableStateFlow` / `StateFlow` : Patrón observable para emitir estados a Compose.
- `viewModelScope` : Coroutine scope vinculado al ciclo de vida del ViewModel.
- `_uiState` (privado, mutable) vs `uiState` (público, solo lectura): Patrón de backing property.

Paso 8: Capa UI - Pantallas

8.1 `ui/screens/pokedex/PokedexScreen.kt`

```
package com.carevalojesus.pokeapi.ui.screens.pokedex

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.grid.GridCells
import androidx.compose.foundation.lazy.grid.LazyVerticalGrid
import androidx.compose.foundation.lazy.grid.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Clear
import androidx.compose.material.icons.filled.Search
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import coil.compose.AsyncImage
import com.carevalojesus.pokeapi.domain.model.PokemonItem

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun PokedexScreen(
    onPokemonClick: (Int) -> Unit,
    viewModel: PokedexViewModel = viewModel()
) {
    val uiState by viewModel.uiState.collectAsState()
    val searchQuery by viewModel.searchQuery.collectAsState()

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Pokédex") },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme
                        .colorScheme.primaryContainer
                )
            )
        }
    ) { padding ->
        Column(
            modifier = Modifier.fillMaxSize().padding(padding)
        ) {
            // --- Barra de búsqueda ---
            OutlinedTextField(
                value = searchQuery,
                onValueChange = {
```

```
            viewModel.onSearchQueryChange(it)
        },
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 8.dp, vertical = 4.dp),
        placeholder = {
            Text("Buscar por nombre o número...")
        },
        leadingIcon = {
            Icon(Icons.Default.Search,
                contentDescription = "Buscar")
        },
        trailingIcon = {
            if (searchQuery.isNotEmpty()) {
                IconButton(onClick = {
                    viewModel.onSearchQueryChange("")
                }) {
                    Icon(Icons.Default.Clear,
                        contentDescription = "Limpiar")
                }
            }
        },
        singleLine = true
    )

    when (val state = uiState) {
        is PokedexUiState.Loading -> {
            Box(
                modifier = Modifier.fillMaxSize(),
                contentAlignment = Alignment.Center
            ) { CircularProgressIndicator() }
        }
        is PokedexUiState.Error -> {
            Box(
                modifier = Modifier.fillMaxSize(),
                contentAlignment = Alignment.Center
            ) {
                Column(
                    horizontalAlignment =
                        Alignment.CenterHorizontally
                ) {
                    Text(state.message,
                        color = MaterialTheme
                            .colorScheme.error)
                    Spacer(Modifier.height(8.dp))
                    Button(onClick = {
                        viewModel.loadPokemonList()
                    }) {
                        Text("Reintentar")
                    }
                }
            }
```

```
                }
                is PokedexUiState.Success -> {
                    if (state.pokemonList.isEmpty()) {
                        Box(
                            modifier = Modifier.fillMaxSize(),
                            contentAlignment = Alignment.Center
                        ) {
                            Text(
                                "No se encontraron resultados",
                                style = MaterialTheme
                                    .typography.bodyLarge,
                                color = MaterialTheme
                                    .colorScheme.onSurfaceVariant
                            )
                        }
                    } else {
                        LazyVerticalGrid(
                            columns = GridCells.Fixed(2),
                            contentPadding = PaddingValues(8.dp),
                            horizontalArrangement =
                                Arrangement.spacedBy(8.dp),
                            verticalArrangement =
                                Arrangement.spacedBy(8.dp)
                        ) {
                            items(state.pokemonList) { pokemon ->
                                PokemonCard(pokemon) {
                                    onPokemonClick(pokemon.id)
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

@Composable
fun PokemonCard(
    pokemon: PokemonItem,
    onClick: () -> Unit
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick),
        elevation = CardDefaults.cardElevation(
            defaultElevation = 4.dp
        )
    ) {
        Column(
            modifier = Modifier.padding(8.dp),
```

```
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            AsyncImage(
                model = pokemon.imageUrl,
                contentDescription = pokemon.name,
                modifier = Modifier.size(120.dp)
            )
            Text(
                "#${pokemon.id}",
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme
                    .colorScheme.onSurfaceVariant
            )
            Text(
                pokemon.name,
                style = MaterialTheme.typography.titleMedium,
                textAlign = TextAlign.Center
            )
        }
    }
}
```

Elementos destacados:

- **Barra de búsqueda**: `OutlinedTextField` con icono de lupa (`Search`) y botón de limpiar (`Clear`) que aparece solo cuando hay texto.
- `LazyVerticalGrid` con 2 columnas para el grid de tarjetas.
- `AsyncImage` (Coil) para carga asíncrona de imágenes.
- `collectAsState()` convierte el `StateFlow` en estado Compose.
- Manejo de los 3 estados: Loading (spinner), Error (mensaje + botón reintentar), Success (grid).
- **Sin resultados**: Cuando el filtro no tiene coincidencias se muestra "No se encontraron resultados".

8.2 `ui/screens/detail/PokemonDetailScreen.kt`

```
package com.carevalojesus.pokeapi.ui.screens.detail

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontStyle
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import coil.compose.AsyncImage
```

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun PokemonDetailScreen(
    pokemonId: Int,
    onBack: () -> Unit,
    viewModel: PokemonDetailViewModel = viewModel()
) {
    LaunchedEffect(pokemonId) {
        viewModel.loadPokemon(pokemonId)
    }
    val uiState by viewModel.uiState.collectAsState()

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Detalle") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(
                            Icons.AutoMirrored.Filled.ArrowBack,
                            contentDescription = "Volver"
                        )
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme
                        .colorScheme.primaryContainer
                )
            )
        }
    ) { padding ->
        when (val state = uiState) {
            is PokemonDetailUiState.Loading -> {
                Box(
                    modifier = Modifier
                        .fillMaxSize().padding(padding),
                    contentAlignment = Alignment.Center
                ) { CircularProgressIndicator() }
            }
            is PokemonDetailUiState.Error -> {
                Box(
                    modifier = Modifier
                        .fillMaxSize().padding(padding),
                    contentAlignment = Alignment.Center
                ) {
                    Text(state.message,
                        color = MaterialTheme.colorScheme.error)
                }
            }
            is PokemonDetailUiState.Success -> {
                val pokemon = state.pokemon
                Column(
```

```
        modifier = Modifier
            .fillMaxSize()
            .padding(padding)
            .verticalScroll(rememberScrollState())
            .padding(16.dp),
        horizontalAlignment =
            Alignment.CenterHorizontally
    ) {
        // Imagen oficial
        AsyncImage(
            model = pokemon.imageUrl,
            contentDescription = pokemon.name,
            modifier = Modifier.size(200.dp)
        )
        Spacer(Modifier.height(16.dp))

        // Nombre en español
        Text(
            "#${pokemon.id} ${pokemon.name}",
            style = MaterialTheme
                .typography.headlineMedium
        )

        // Categoría (genus)
        if (pokemon.genus.isNotEmpty()) {
            Text(
                pokemon.genus,
                style = MaterialTheme
                    .typography.bodyMedium,
                fontStyle = FontStyle.Italic,
                color = MaterialTheme
                    .colorScheme.onSurfaceVariant
            )
        }

        Spacer(Modifier.height(8.dp))

        // Tipos en español
        Row(horizontalArrangement =
            Arrangement.spacedBy(8.dp)
        ) {
            pokemon.types.forEach { type ->
                AssistChip(
                    onClick = {},
                    label = { Text(type) }
                )
            }
        }

        // Descripción de la Pokédex
        if (pokemon.description.isNotEmpty()) {
            Spacer(Modifier.height(12.dp))
```

```
                    Card(elevation =
                        CardDefaults.cardElevation(
                            defaultElevation = 1.dp
                        )
                    ) {
                        Text(
                            pokemon.description,
                            modifier = Modifier
                                .padding(12.dp),
                            style = MaterialTheme
                                .typography.bodyMedium,
                            textAlign = TextAlign.Justify
                        )
                    }
                }

                Spacer(Modifier.height(16.dp))

                // Altura y peso
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement =
                        Arrangement.SpaceEvenly
                ) {
                    InfoCard("Altura",
                        "${pokemon.height / 10.0} m")
                    InfoCard("Peso",
                        "${pokemon.weight / 10.0} kg")
                }

                Spacer(Modifier.height(16.dp))

                // Estadísticas base en español
                Text(
                    "Estadísticas base",
                    style = MaterialTheme
                        .typography.titleLarge
                )
                Spacer(Modifier.height(8.dp))

                pokemon.stats.forEach { stat ->
                    StatBar(
                        name = stat.name,
                        value = stat.value
                    )
                    Spacer(Modifier.height(4.dp))
                }
            }
        }
    }
}
}
```

```
@Composable
fun InfoCard(label: String, value: String) {
    Card(elevation =
        CardDefaults.cardElevation(defaultElevation = 2.dp)
    ) {
        Column(
            modifier = Modifier.padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(label,
                style = MaterialTheme.typography.bodySmall)
            Text(value,
                style = MaterialTheme.typography.titleMedium)
        }
    }
}

@Composable
fun StatBar(name: String, value: Int) {
    Row(
        modifier = Modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(
            text = name,
            modifier = Modifier.width(72.dp),
            style = MaterialTheme.typography.bodyMedium
        )
        Text(
            text = "$value",
            modifier = Modifier.width(36.dp),
            style = MaterialTheme.typography.bodyMedium
        )
        LinearProgressIndicator(
            progress = { value / 255f },
            modifier = Modifier
                .weight(1f)
                .height(8.dp),
        )
    }
}
```

Elementos destacados:

- `LaunchedEffect(pokemonId)` dispara la carga cuando cambia el ID.
- **Nombre en español** obtenido del endpoint `pokemon-species`.
- **Categoría** mostrada en cursiva debajo del nombre (ej: "Pokémon Semilla").
- **Descripción de la Pokédex** en español dentro de una Card.
- **Tipos en español**: "Fuego", "Agua", "Planta", etc.
- **Estadísticas en español**: "PS", "Ataque", "Defensa", "At. Esp.", "Def. Esp.", "Velocidad".

- `StatBar` usa directamente los nombres traducidos por el repositorio (ancho de 72dp para acomodar las etiquetas en español).
- `LinearProgressIndicator` para las barras de stats (rango 0-255).
- Conversión de unidades: height/10 = metros, weight/10 = kilogramos.

Paso 9: Navegación

9.1 `ui/navigation/AppNav.kt`

```
package com.carevalojesus.pokeapi.ui.navigation

import androidx.compose.runtime.Composable
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.carevalojesus.pokeapi.ui.screens.detail
    .PokemonDetailScreen
import com.carevalojesus.pokeapi.ui.screens.pokedex
    .PokedexScreen

@Composable
fun AppNav() {
    val navController = rememberNavController()

    NavHost(
        navController = navController,
        startDestination = "pokedex"
    ) {
        composable("pokedex") {
            PokedexScreen(
                onPokemonClick = { id ->
                    navController.navigate("detail/$id")
                }
            )
        }
        composable(
            route = "detail/{pokemonId}",
            arguments = listOf(
                navArgument("pokemonId") {
                    type = NavType.IntType
                }
            )
        ) { backStackEntry ->
            val pokemonId = backStackEntry
                .arguments?.getInt("pokemonId") ?: 1
            PokemonDetailScreen(
                pokemonId = pokemonId,
                onBack = { navController.popBackStack() }
```

```
            )
        }
    }
}
```

Conceptos clave:

- `NavHost` define el contenedor de navegación con un destino inicial (`"pokedex"`).
- `navArgument` define parámetros tipados en la ruta (`{pokemonId}` a `Int`).
- `navController.navigate("detail/$id")` navega pasando el ID.
- `navController.popBackStack()` regresa a la pantalla anterior.

Paso 10: MainActivity

```
package com.carevalojesus.pokeapi

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.carevalojesus.pokeapi.ui.navigation.AppNav
import com.carevalojesus.pokeapi.ui.theme.PokeApiTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            PokeApiTheme {
                AppNav()
            }
        }
    }
}
```

`MainActivity` solo inicializa el tema y delega toda la UI al grafo de navegación `AppNav()` .

Flujo de datos completo

```
PokéAPI (JSON)
    ↓  Retrofit + Gson
PokemonApi (suspend functions)
    ├─ pokemon/{id}       → PokemonDetailResponse
    └─ pokemon-species/{id} → PokemonSpeciesResponse
                ↓  (parallel con async)
PokemonRepository
    ├─ DTO → Domain Model
    ├─ typeTranslations (Inglés → Español)
    ├─ statTranslations (Inglés → Español)
```

```
    └─ species → nombre, categoría, descripción (es)
                ↓
ViewModel (StateFlow<UiState>)
                ↓  collectAsState()
Composable Screen (UI en español)
```

1. El **ViewModel** llama al **Repository** dentro de `viewModelScope.launch`.
2. El **Repository** usa `async` para ejecutar en paralelo las peticiones a `pokemon/{id}` y `pokemon-species/{id}`.
3. **Gson** deserializa el JSON en **DTOs**.
4. El **Repository** transforma los DTOs en modelos de dominio, traduciendo tipos y stats al español y extrayendo nombre/categoría/descripción en español de la respuesta de species.
5. El **ViewModel** actualiza el `StateFlow` con el nuevo estado.
6. La **pantalla Compose** recompone automáticamente al detectar el cambio.

Resumen de librerías utilizadas

Librería	Versión	Propósito
Retrofit	2.11.0	Cliente HTTP para consumir la API REST
Converter Gson	2.11.0	Serialización/deserialización JSON a Kotlin
Coil Compose	2.7.0	Carga asíncrona de imágenes en Compose
Navigation Compose	2.8.4	Navegación entre pantallas en Compose
Lifecycle ViewModel	2.10.0	ViewModel con integración Compose
Material 3	BOM 2024.09	Componentes de UI (Cards, Chips, Bars, etc.)

Resumen de endpoints utilizados

Endpoint	Método	Uso
`/api/v2/pokemon?limit=151`	GET	Lista de los 151 Pokémon originales
`/api/v2/pokemon/{id}`	GET	Datos técnicos: tipos, stats, sprites, peso, altura
`/api/v2/pokemon-species/{id}`	GET	Datos localizados: nombre, categoría y descripción en español

---

Ejercicios propuestos

1. **Paginación**: Modificar el repositorio para cargar Pokémon de 20 en 20 conforme el usuario hace scroll.
2. **Favoritos**: Implementar persistencia local con Room para guardar Pokémon favoritos.
3. **Color por tipo**: Asignar un color de fondo a cada `PokemonCard` según su tipo principal (Fuego a rojo, Agua a azul, etc.).
4. **Animaciones**: Agregar transiciones animadas entre pantallas usando `AnimatedNavHost`.
5. **Búsqueda avanzada**: Extender el buscador actual para filtrar también por tipo (ej: escribir "fuego" muestre todos los Pokémon de tipo Fuego).